

NAME: ADEBIMPE ADEROJU M.

COURSE: CSE210- PROGRAMMING WITH FUNCTIONS

INSTRUCTURE'S NAME: JOHN READING

WHAT IS ENCAPSULATION AND WHY IS IT IMPORTANT?

Encapsulation is one of the core principles of object-oriented programming that bundles data (fields) and the methods that operate on that data into a single unit called a class, while restricting direct access to some of an object's components. This means we hide the internal details of how a class works and only expose what users need through controlled interfaces like public methods. A key benefit is data protection—private fields can't be accidentally modified by other parts of the program, which prevents bugs and makes code more reliable.

For example, in my Scripture Memorizer program, each Word object manages its own hidden/shown state internally. Here's the code that demonstrates encapsulation:

```
public class Word
{
    private readonly string _originalText; // Private - hidden from outside
    private bool _isHidden;                // Private - internal state

    public bool IsHidden => _isHidden;     // Public read-only access
    public void Hide()                    // Public controlled method
    {
        _isHidden = true;
    }
}
```

Here, **_originalText** and **_isHidden** are private, so no one outside the **Word** class can directly change them. Instead, they must use the public **Hide()** method or read **IsHidden** property. This ensures that when a word gets hidden, it follows the correct process (setting underscores matching the letter count) and can't be corrupted. The **Scripture** class can reliably hide words knowing each **Word** object properly manages its own state.

This approach makes the code modular—each class handles its own responsibilities without interfering with others. If I needed to change how hiding works (say, adding animation), I could modify just the Word class without breaking the entire program. Encapsulation creates cleaner, more maintainable code that's easier to debug and extend, which is why it's so valuable in real-world software development.